

Innovation Document

Movie App — Mobile Applications Development Project

Overview

This document describes the additional features implemented beyond the requirements specified in the project brief. Each feature is described in terms of what it does, how it was implemented, and why it adds value to the user experience. The goal across all of them was the same: make the app feel finished rather than functional, and bring it closer to the expectations users have from production-grade mobile apps.

1. Pull-to-Refresh on Home and Favourites Pages

What it does

Both the Home Page and Favourites Page support the standard pull-down gesture familiar from native mobile apps. Dragging the content downward at the top of the screen displays a small spinner, refreshes the list, and dismisses the spinner once the new data is in place.

How it works

Implemented using Ionic's **ion-refresher** component, placed at the top of **ion-content** with `slot="fixed"`. The `ionRefresh` event triggers a handler that re-fetches data — a network call to the TMDB trending endpoint on the Home Page, or a synchronous re-read from `localStorage` on the Favourites Page. In both cases the handler calls `event.target.complete()` to dismiss the refresher's built-in spinner once the list has been updated.

Why it matters

Pull-to-refresh has become an expected gesture across mobile apps. Adding it costs very little code but signals to users that the app behaves the way they expect it to. It also gives users a way to manually update trending data, which would otherwise only refresh when the app is relaunched.

2. Infinite Scroll for Trending, Search, and Genre Results

What it does

The Home Page now loads movies progressively as the user scrolls down. When the user reaches the bottom of the list, the next page of results is fetched from TMDB and appended to the existing list. This works transparently across all three list sources: trending, search results, and genre-filtered results.

How it works

Implemented using Ionic's **ion-infinite-scroll** component combined with TMDB's `page` query parameter. The relevant service methods (`getTrendingMovies`, `searchMovies`, `getMoviesByGenre`) were extended to accept a page number, defaulting to 1 for backwards compatibility. The Home Page tracks the current page, total pages, and active mode in component state. When `ionInfinite` fires, `loadMore()` picks the correct service method based on the active mode, fetches the next page, and appends results to the existing list. The scroller automatically disables itself once the final page is reached, and re-enables on every fresh load (new search, new genre, or reset to

trending).

Why it matters

TMDB returns thousands of pages of trending and search data, but the baseline implementation would only ever show the first 20 movies. Infinite scroll unlocks the rest of the catalogue without overwhelming the user, and avoids any explicit pagination UI that would feel dated. The implementation also demonstrates handling state coordination across multiple list sources, which was the most architecturally interesting part of the project.

3. Genre Filtering on the Home Page

What it does

A genre dropdown was added above the search bar on the Home Page. Selecting a genre — Action, Comedy, Horror, etc. — replaces the current movie list with films matching that genre. Selecting "All genres" resets the list back to today's trending movies.

How it works

Two TMDB endpoints were added to the service layer: `/genre/movie/list` fetches the master list of genre IDs and names once when the page loads, and `/discover/movie?with_genres=<id>` fetches movies for the selected genre. A new *Genre* interface was added to the model file, and a *getGenres* and *getMoviesByGenre* method were added to the TMDB service. The dropdown uses Ionic's **ion-select** component bound via *ngModel*, with an *ionChange* handler that updates the displayed list. Genre, search, and trending modes are mutually exclusive: picking one clears the others, so the active filter and the section heading always stay in sync.

Why it matters

TMDB exposes a rich genre taxonomy that was otherwise invisible to users of the app. Filtering by genre is one of the most common ways people browse film catalogues, and adding it makes the app meaningfully more useful for movie discovery rather than just lookup. Coordinating the three list sources (trending, search, genre) without letting them conflict required careful state management, which was a useful exercise in keeping UI state consistent.

4. Placeholder for Missing Movie Posters

What it does

Movies returned by TMDB occasionally have no poster image — particularly older or more obscure films. Rather than rendering a broken image icon or leaving an awkward gap in the layout, the app now displays a neutral placeholder image for any movie whose *poster_path* is null.

How it works

A placeholder image was added to *src/assets/images/no-poster.png*. The *getImageUrl* helper in the TMDB service was changed to return the placeholder path whenever the input *poster_path* is null or empty, instead of returning null itself. As a result, every poster *img* tag across the Home, Movie Details, and Favourites pages now always renders a valid image, removing the need for conditional **ngIf* guards on each image element.

Why it matters

Visual consistency matters for list-based UIs. With the placeholder, every row in a movie list has the same shape regardless of whether TMDb has a poster on file. Without it, missing posters cause irregular layouts that look like layout bugs rather than missing data. Centralising the fallback in the service also means there is a single place to update if the placeholder design ever changes.

5. "No Image" Fallback for Cast and Crew Photos

What it does

On the Details Page, when a cast or crew member has no profile photo available, the app displays the literal text *No image* in place of the missing portrait. This handles a small but visible edge case in the TMDb dataset.

How it works

The Details Page template uses Angular's **ng-template** with an *else* clause on the profile image's **ngIf*. When *person.profile_path* is truthy, the image is rendered; otherwise the named template *#noImage* renders a styled paragraph with the fallback text. The same pattern is applied to the conditional rendering of birthday, deathday, and "also known as", each of which is hidden when the corresponding TMDb field is empty.

Why it matters

This requirement is described in the project brief (Figure 7), which shows partial-information records as deliberately incomplete rather than padded with placeholder data. The implementation respects that design intent: real fields only render when the underlying data exists, and the missing-image case is communicated clearly to the user rather than hidden.

6. Loading States Across Asynchronous Pages

What it does

Every page that fetches data from the TMDb API now shows a loading spinner while the request is in flight. This applies to the Home Page (initial trending load and search), the Movie Details Page (credits fetch), and the Details Page (parallel person and credits fetches). The Favourites Page deliberately omits this, since its data source is synchronous and instant.

How it works

Each affected page tracks a **loading** boolean in component state. The flag is set to true when an API call begins and flipped to false in the *subscribe* callback once the response arrives (or in the *catchError* branch on failure). The template uses **ngIf="loading"* to render an **ion-spinner** while the flag is true, and **ngIf="!loading"* on the corresponding list to ensure only one of the two renders at any given time. The Details Page uses the same pattern around its *forkJoin*, which fires both API requests in parallel and resolves the spinner when both complete.

Why it matters

Without explicit loading feedback, an empty page during a slow API call looks broken rather than busy. The spinner gives users a reliable signal that the app is working, which becomes especially important on slower networks or when the TMDb API is under load. The deliberate exclusion of the spinner from the Favourites Page reflects an understanding that synchronous reads from *localStorage* do not need loading states — adding one would just produce a one-frame flash that

looked like a glitch.

Summary

Together these six features push the app beyond a literal reading of the brief and toward something closer to a production-quality mobile experience. Pull-to-refresh and infinite scroll bring the app in line with native gesture expectations. Genre filtering meaningfully expands how users can explore the TMDb catalogue. Placeholders, fallbacks, and loading states address the edge cases and asynchronous moments that distinguish a polished app from one that only handles the happy path. Each feature was chosen because it adds visible user value and demonstrates a different aspect of the framework — lifecycle hooks, RxJS operators, component composition, and state coordination — rather than just adding superficial polish.